
```
oStack.push(<someData>)
```

And it would retrieve the most recent data by calling a `pop()` method like this:

```
<someVariable> = oStack.pop()
```

The client would not need to know or care how these methods were implemented or how the data was stored. The implementation of the `Stack` would be handled completely by the methods of the `Stack`.

While the client code could view a `Stack` class as a black box, writing such a class in Python is fairly trivial. [Listing 8-8](#) shows how it could be implemented.

File: Stack/Stack.py

```
# Stack class

class Stack():
    ''' Stack class implements a last in first out LIFO algorithm'''
    def __init__(self, startingStackAsList=None):
        if startingStackAsList is None:
            ❶ self.dataList = [ ]
        else:
            self.dataList = startingStackAsList[:] # make a copy

    ❷ def push(self, item):
        self.dataList.append(item)

    ❸ def pop(self):
        if len(self.dataList) == 0:
            raise IndexError
        element = self.dataList.pop()
        return element

    ❹ def peek(self):
        # Retrieve the top item, without removing it
        item = self.dataList[-1]
        return item

    ❺ def getSize(self):
```